🎓 Projeto Final de Arquitetura Web: "O Sistema Full Stack"

Grupos: Duplas

1. 🎯 O Objetivo

O objetivo deste trabalho é desenvolver uma aplicação web completa (Full Stack), aplicando os conceitos de arquitetura em camadas (MVC), comunicação cliente-servidor, persistência de dados NoSQL e segurança.
A aplicação deve ser dividida em duas grandes estruturas arquiteturais:
Backend (API): O cérebro e o cofre dos dados.
Frontend (SPA): A interface rica e responsiva.

2. 🏗️ Requisitos Técnicos Obrigatórios

- A. Backend (Node.js + Express + MongoDB)
  Arquitetura: Deve seguir o padrão MVC (Model, View*, Controller), mas adaptado para API.
  Nota: Como o frontend é separado, a "View" do backend será apenas a entrega de dados em formato JSON.
  Banco de Dados: Utilizar MongoDB com Mongoose.
  Operações: Implementar o CRUD completo (Create, Read, Update, Delete) para pelo menos duas entidades relacionadas (Ex: Usuários e Produtos).
  Segurança (AuthN): Sistema de Login. A senha deve ser armazenada com Hash (bcrypt).
  Controle de Acesso (AuthZ): Implementar pelo menos 2 níveis de acesso (Ex: Admin e UsuarioComum).
  Exemplo: Apenas Admins podem deletar itens ou ver o dashboard.
  Validação: O backend não pode confiar nos dados do frontend. Deve haver validação na entrada (ex: Joi ou Mongoose validation).

- B. Frontend (React.js)
  Tecnologia: React (pode usar Vite ou Create React App).
  Consumo de API: Usar fetch ou axios para comunicar com o seu Backend.
  Roteamento: Usar react-router-dom para navegação sem recarregar a página (SPA).
  Gestão de Estado: Gerenciar o estado do usuário logado (saber se está logado ou não e qual o nível de acesso).
  Responsividade: O layout deve se adaptar a Desktop e Mobile (uso de CSS Flexbox/Grid ou frameworks como Tailwind/Bootstrap é permitido).
  Usabilidade:
  - Feedback visual de erros (ex: "Senha incorreta").
  - Feedback de carregamento (spinners/skeletons enquanto a requisição acontece).

3. 💡 Sugestões de Temas (Escolha Livre)

As duplas podem escolher um tema livre, desde que atenda aos requisitos técnicos. Aqui estão 3 sugestões com complexidade equilibrada:

- Opção 1: "E-commerce Geek" (Loja Virtual)
  Entidades: Usuários, Produtos, Carrinho/Pedidos.
  Fluxo: Usuários veem produtos e compram. Admins cadastram produtos e veem total de vendas.
  Desafio de UX: Galeria de imagens dos produtos e filtro de categorias.

- Opção 2: "Rede Social Acadêmica" (Fórum/Feed)
  Entidades: Usuários, Postagens, Comentários.
  Fluxo: Usuários postam dúvidas e comentam. Moderadores (Admins) podem excluir posts ofensivos.
  Desafio de UX: Feed infinito ou paginação e atualização em tempo real (bônus).

- Opção 3: "Sistema de Agendamento" (Barbearia/Consultório/Labs da UFRR)
  Entidades: Clientes, Serviços, Agendamentos.
  Fluxo: Cliente escolhe data/hora e serviço. Admin gerencia a agenda e disponibilidade.
  Desafio de UX: Calendário visual e bloqueio de horários já ocupados.

4. 🚀 Diferenciais (Bônus na Nota)

Se o grupo quiser ir além (excelente para o portfólio):
WebSockets (Socket.io): Implementar uma funcionalidade em tempo real (ex: chat de suporte, notificação de novo pedido, atualização de estoque ao vivo).
Deploy na Nuvem: Colocar o Backend (ex: Render/Railway) e o Frontend (ex: Vercel/Netlify) no ar.
Testes: Implementar testes unitários básicos (Jest) em pelo menos uma rota crítica.

5. 📦 O que deve ser entregue?

Link do Repositório (GitHub):
O repositório deve ter duas pastas claras: /server (backend) e /client (frontend), ou dois repositórios separados.
README.md: Deve conter instruções de como rodar o projeto localmente (npm install, npm start, variáveis de ambiente necessárias).
Apresentar em sala de aula o sistema funcionando (3-5 min):
Mostrando o fluxo de cadastro, login, erro de login, CRUD funcionando e a diferença de acesso entre Admin e Usuário.

6. 📊 Critérios de Avaliação

Critério Peso Detalhes
Arquitetura Backend 30% Organização MVC, uso correto dos verbos HTTP, conexão Mongo.

Arquitetura Frontend 30% Componentização React, consumo da API, Roteamento.
Segurança 20% Login funcional, proteção de rotas (Backend) e Hash de senha.
UX/UI 10% Responsividade, feedback de erros, design agradável.
Boas Práticas 10% Código limpo, uso do Git (commits semânticos), organização.